“Multi-Process Task Manager for Linux”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

B. DEEPAK GOUD	**2520080035**
A. LAKSHMI SAI TEJA	**2520080005**
E. PRANAY	**2520080061**

Under the Esteemed Guidance of

Dr. M. Raghupati
[Designation]
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Multi-Process Task Manager for Linux"** is a record of Bonafide work of **B. Deepak Goud – 2520080035, A. Lakshmi Sai Teja – 2520080005, E. Pranay – 2520080061** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The work reported here was carried out by us under the guidance of Dr. M. Raghupati. The results embodied in this report have not been copied from any other departments/University/Institute.

B. DEEPAK GOUD	**2520080035**
A. LAKSHMI SAI TEJA	**2520080005**
E. PRANAY	**2520080061**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the project based report entitled **"Multi-Process Task Manager for Linux"** is a bonafide work done and submitted by **B. Deepak Goud – 2520080035, A. Lakshmi Sai Teja – 2520080005, E. Pranay – 2520080061,**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **[20XX–20XX]**. The work was carried out under the guidance of **Dr. M. Raghupati**, [Designation], Department of Computer Science and Engineering.

Signature of the Guide
(Dr. M. Raghupati)

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We, the team members, express our sincere thanks to all those who assisted us in one way or the other in the completion of this project.

We express our greatest appreciation to our project guide, Dr. M. Raghupati, [Designation], Department of Computer Science and Engineering, and to our Course Coordinator, [Name of Course Coordinator], for their tremendous support, encouragement and guidance throughout this project.

We express our gratitude to **Dr. P. Venkateshwara Rao (CSE)**, Head of the ***Department of Computer Science Engineering***, for providing us with the facilities and the resources needed to complete this project-based lab.

We also thank K L (Deemed to be) University, the teaching and non-teaching staff of the department, and all those who helped us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

B. DEEPAK GOUD **2520080035**

A. LAKSHMI SAI TEJA **2520080005**

E. PRANAY **2520080061**

Abstract

Monitoring and controlling processes is a basic task in Linux system administration, yet it usually requires several separate commands such as ps, top, grep and kill, each with its own options and output format. This project, the Multi-Process Task Manager for Linux, addresses the problem by providing a lightweight, terminal-based task manager written in C that brings process listing, searching, resource monitoring and control into one menu-driven interface.

The application obtains real process information directly from the Linux /proc pseudo-filesystem. It discovers running processes by scanning the numeric directories of /proc and reads each process's PID, parent PID, name and state from /proc/[PID]/stat. CPU usage is calculated from two successive samples of a process's user and kernel time together with the system-wide totals in /proc/stat, while memory usage is derived from the resident set size (VmRSS) and the total RAM reported in /proc/meminfo. The user can search for a process, select it and control it by sending the SIGTERM or SIGKILL signal through the kill() system call, with input validation and explicit handling of permission errors, missing processes and race conditions.

The project is implemented in C and compiled with GCC on Ubuntu Linux, using Make for the build process and Git and GitHub for version control; it requires no external libraries. It demonstrates core Operating Systems concepts such as processes, process states, CPU accounting, virtual memory and signals. Its practical applications include teaching, development and testing, and simple system troubleshooting on machines without a graphical environment.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		9–11
		1.1 Background of the Project	9
		1.2 Problem Statement	9
		1.3 Scope of the Project	10
		1.4 Objectives of the Project	10
		1.5 Importance of the Application	11
		1.6 Target Users and Audience	11
2	OS Concepts and Background Study		12–16
		2.1 Operating System Concepts Related to the Project	12
		2.2 Linux Process Management	12
		2.3 Process IDs, Parent Process IDs, and Process States	12
		2.4 The Linux /proc Pseudo-Filesystem	13
		2.5 CPU Scheduling and CPU Usage Accounting	13
		2.6 Virtual Memory, Resident Memory, and RAM Utilization	14
		2.7 Linux Signals and Process Termination	14
		2.8 Existing Methods and Tools, such as ps, top, and htop	15
		2.9 Proposed Approach	15
		2.10 Advantages of the Proposed Approach	15
3	System Requirements		17–19
		3.1 Hardware Requirements	17
		3.2 Software Requirements	17
		3.3 Operating System Environment — Ubuntu Linux	17
		3.4 C Compiler and GCC	18
		3.5 Development Tools	18
		3.6 Minimum System Requirements	18
		3.7 Functional Requirements	18
		3.8 Non-Functional Requirements	19
4	Technology and Tools Used		20–22
		4.1 C Programming Language	20

		4.2 GCC Compiler	**20**
		4.3 Linux /proc Filesystem	**20**
		4.4 Linux System Calls and APIs	**20**
		4.5 Signals — SIGTERM and SIGKILL	**21**
		4.6 Process Information Files	**21**
		4.7 Linux Commands and Utilities	**21**
		4.8 Makefile and Build Process	**21**
		4.9 Git and GitHub	**22**
		4.10 Other Libraries or Tools Actually Used	**22**
5	**System Architecture and Design**		**23–29**
		5.1 Overall System Architecture	**23**
		5.2 System Workflow	**23**
		5.3 Architecture Diagram	**24**
		5.4 Data Flow Diagram	**24**
		5.5 Process Monitoring Flowchart	**25**
		5.6 Process Termination Flowchart	**26**
		5.7 Module Description	**27**
		5.8 Input and Output Design	**28**
		5.9 Error Handling and Permission Management	**28**
		5.10 Security Considerations	**29**
6	**Implementation**		**30–34**
		6.1 Project Directory Structure	**30**
		6.2 Module-Wise Implementation	**30**
		6.3 Process Discovery Using /proc	**30**
		6.4 Reading Process Information	**30**
		6.5 CPU Usage Calculation	**31**
		6.6 Memory Usage Calculation	**31**
		6.7 Process Search and Selection	**31**
		6.8 Safe Process Termination	**32**
		6.9 Linux System Calls and APIs Used	**32**
		6.10 Input Validation and Error Handling	**33**
		6.11 Algorithms and Pseudocode	**33**
		6.12 Core C Code Explanation	**33**
		6.13 Compilation and Build Process	**34**
7	**Execution and Demonstration**		**35–38**
		7.1 Ubuntu Environment Setup	**35**

		7.2 Compilation Using GCC or Makefile	**35**
		7.3 Execution Commands	**35**
		7.4 Application Menu	**35**
		7.5 Displaying Running Processes	**36**
		7.6 Searching for a Process	**36**
		7.7 Viewing CPU and Memory Usage	**36**
		7.8 Terminating a Selected Test Process	**37**
		7.9 Error Handling Demonstration	**37**
		7.10 Screenshots of Execution	**38**
		7.11 Demonstration of Operating Systems Concepts	**38**
8	**Testing and Results**		**39–41**
		8.1 Testing Methodology	**39**
		8.2 Functional Testing	**39**
		8.3 CPU Usage Calculation Testing	**39**
		8.4 Memory Usage Testing	**39**
		8.5 Process Control Testing	**39**
		8.6 Invalid Input Testing	**39**
		8.7 Permission Error Testing	**39**
		8.8 Process Disappearance and Race Condition Testing	**39**
		8.9 Test Case Table	**40**
		8.10 Results and Discussion	**40**
		8.11 Limitations	**41**
9	**Conclusion and Future Enhancements**		**42–43**
		9.1 Project Summary	**42**
		9.2 Objectives Achieved	**42**
		9.3 Operating Systems Learning Outcomes	**42**
		9.4 Benefits of the Application	**42**
		9.5 Current Limitations	**42**
		9.6 Future Enhancements	**43**
10	**References**		**44–45**
		10.1 Books	**44**
		10.2 Linux Manual Pages	**44**
		10.3 GNU and Kernel Documentation	**44**
		10.4 Online Resources	**45**

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

An operating system (OS) is the system software that manages hardware resources and provides services to application programs. One of its most important responsibilities is **process management**. A process is a program in execution, and at any moment a Linux system may be running dozens or hundreds of them: the desktop environment, background services, network daemons, terminals, editors and user applications. The kernel creates these processes, schedules them on the available CPU cores, allocates memory to them and finally removes them when they finish.

Linux makes the internal state of the kernel visible to ordinary programs through the **`/proc` pseudo-filesystem**. Every running process has a directory named after its Process ID (PID), such as `/proc/1234`, containing small text files that describe the process: its name, state, parent, CPU time consumed and memory usage. Because these are ordinary files, a program written in C can read them with standard file functions and build its own picture of the system without any special library.

This project, the **Multi-Process Task Manager for Linux**, uses that facility to build a lightweight, terminal-based task manager in the C programming language. The application reads real process information from `/proc`, calculates CPU and memory usage, and lets the user select a process and terminate it using Linux signals. The word “multi-process” in the title refers to the fact that the tool monitors and manages many processes at the same time; it does not mean that the tool itself creates child processes.

1.2 Problem Statement

The problem statement given for this project is: **“Develop a lightweight task manager that displays process information and allows users to control selected processes.”**

On a typical Linux system, a user who wants to find out what is running and then stop a misbehaving program must combine several separate tools. The `ps` command prints a snapshot of processes, `top` or `htop` shows live usage figures, `grep` is needed to search the output, and `kill` is used to send a signal, usually after the user has copied the PID by hand. This workflow has several difficulties:

- **Fragmented tools:** listing, searching, measuring and terminating are four different commands with different options and output formats.
- **Manual PID handling:** the PID must be read from one output and typed into another command. A single wrong digit can terminate an unrelated process.

- **Hard-to-read output for beginners:** the default output of `ps` and `top` contains many columns and options that a first-time user must memorise.
- **No guided error messages:** raw command-line errors such as “Operation not permitted” or “No such process” are not explained to the user.
- **Limited insight into how the data is obtained:** the commands hide the kernel interfaces they use, so students do not see how monitoring actually works.

The project therefore aims to provide **one unified interface** in which the user can view processes, search for one, inspect its CPU and memory usage, and control it, while the program validates input and reports errors in plain language.

1.3 Scope of the Project

The scope of the project covers the following functions:

- Discovering all running processes by scanning the numeric directories of `/proc`.
- Displaying per-process information: PID, parent PID (PPID), process name and process state.
- Calculating and displaying per-process CPU usage from successive samples of `/proc/[PID]/stat` and `/proc/stat`.
- Displaying per-process resident memory (`VmRSS`) and its percentage of total RAM obtained from `/proc/meminfo`.
- Searching for and selecting a process by PID or by name.
- Controlling a selected process by sending `SIGTERM` or `SIGKILL` through the `kill()` system call, with input validation and error handling.

The following are **outside** the scope of the current version and are listed as future work in Chapter 9: a graphical user interface, historical usage graphs, sorting and filtering options, configurable refresh intervals, resource alerts, process-tree visualisation, disk and network monitoring, changing process priorities, and support for operating systems other than Linux.

Note to authors (delete before submission): *Scope items above follow the project brief. Confirm each one against the final source code before submission and move any item that is not implemented into the future-work list.*

1.4 Objectives of the Project

1. To study how the Linux kernel represents processes and exposes their information through `/proc`.
2. To design and implement a lightweight task manager in C that reads real process data instead of using simulated values.
3. To calculate CPU usage correctly from two samples of cumulative CPU time, and

memory usage from `VmRSS` and `MemTotal`.

4. To allow a user to search for and select a process and to terminate it safely using the `SIGTERM` and `SIGKILL` signals.
5. To handle errors such as invalid input, insufficient permissions and processes that exit while being read.
6. To demonstrate Operating Systems concepts (processes, scheduling, virtual memory and signals) through a working program.
7. To build and document the project using standard Linux tools: GCC, Make, and Git.

1.5 Importance of the Application

Process management is a core topic of every Operating Systems course, but it is often studied only through theory. A task manager that works on real kernel data connects the concepts of the PCB (process control block), process states, CPU time accounting, resident memory and signals with observable behaviour. For a user, the application shortens the path from "something is slow" to "this process is the cause" and offers a single, safe place to act on that finding. Because the program is written in C and depends only on the standard C library and the `/proc` interface, it is small, has no external dependencies and can run on any Linux machine, including servers where no graphical environment is installed.

1.6 Target Users and Audience

- **Undergraduate students** learning Operating Systems and Linux system programming.
- **Beginner Linux users** who want a simpler alternative to remembering several commands.
- **Developers and testers** who need to find and stop runaway test programs on a development machine.
- **Teachers and evaluators** who want a clear example of how user-space programs read kernel-provided information.

CHAPTER 2: OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The project touches several fundamental Operating Systems topics. Table 2.1 summarises how each concept appears in the application.

Table 2.1: Operating Systems concepts and their role in the project

OS Concept	Role in the Project
Process and PCB	Each entry in the task manager corresponds to one process; the data shown (PID, PPID, state, CPU time) comes from the kernel's per-process bookkeeping.
Process states	The state letter (R, S, D, Z, T) read from `/proc/[PID]/stat` is displayed to the user.
CPU scheduling and accounting	Cumulative CPU ticks recorded by the scheduler are sampled twice to compute usage.
Virtual memory	Resident set size (`VmRSS`) shows how much physical RAM a process currently occupies.
Inter-process signalling	`SIGTERM` and `SIGKILL` are delivered with `kill()` to request or force termination.
Protection and permissions	A normal user can signal only their own processes; the kernel returns `EPERM` otherwise.
System calls / user-kernel boundary	File I/O on `/proc` and the `kill()` call are the interfaces between user space and the kernel.

An important distinction must be made. This application **monitors existing processes**: it observes processes that were already created by the kernel or by other programs and never creates new ones. **Creating processes** is a different topic, done with `fork()`, `exec()` and `wait()`. The task manager does not use these calls; it only reads information and, on request, sends a signal to an existing process.

2.2 Linux Process Management

In Linux the kernel represents each process by a `task_struct` data structure, which is the Linux implementation of the process control block. It holds the PID, state, scheduling information, memory map, open files, signal handlers and accounting data. Processes form a tree: every process except the first one (PID 1, usually `systemd` on Ubuntu) has a parent that created it. When a process terminates, the kernel keeps a small record until the parent collects its exit status with `wait()`. Threads in Linux are also scheduled as tasks; the `/proc` directory lists the thread-group leader as the process and the other threads under `/proc/[PID]/task/`.

2.3 Process IDs, Parent Process IDs, and Process States

A **PID** is a positive integer that uniquely identifies a process while it is alive. The **PPID** is the PID of its parent. If a parent exits first, its children are adopted by an ancestor reaper (typically

PID 1), so their PPID changes. PIDs are allocated in increasing order and wrap around when the limit in `/proc/sys/kernel/pid_max` is reached; a PID can therefore be **reused** by a new process after the old one has exited. This matters for a task manager because a PID that was valid when the user selected it may refer to a different process a moment later (see Section 6.8).

Each process is always in one state. The single-letter codes reported by the kernel are given in Table 2.2.

Table 2.2: Linux process state codes (as documented in proc(5))

Code	State	Meaning
R	Running	Running on a CPU or ready to run in a run queue
S	Sleeping	Interruptible sleep: waiting for an event such as I/O or a timer
D	Disk sleep	Uninterruptible sleep, usually waiting for I/O; cannot be interrupted by signals
T / t	Stopped	Stopped by a job-control signal or by a tracer/debugger
Z	Zombie	Terminated but not yet reaped by its parent
I	Idle	Idle kernel thread
X	Dead	Should never be seen in normal operation

2.4 The Linux /proc Pseudo-Filesystem

`/proc` is a **virtual filesystem**: its files are not stored on a disk but generated by the kernel at the moment they are read. It acts as a window into kernel data structures. The files that matter for this project are listed in Table 2.3.

Table 2.3: /proc files used by the task manager

File	Information obtained
`/proc/` (directory)	One numeric sub-directory per running process; scanning it discovers all PIDs.
`/proc/[PID]/stat`	Single-line record: PID, command name, state, PPID, `utime`, `stime`, start time, virtual size, resident pages, etc.
`/proc/[PID]/status`	Human-readable key/value lines including `Name`, `State`, `PPid`, `Uid`, `VmSize` and `VmRSS`.
`/proc/stat`	System-wide cumulative CPU time in clock ticks (user, nice, system, idle, iowait, irq, softirq, steal, ...).
`/proc/meminfo`	System-wide memory figures in kilobytes: `MemTotal`, `MemFree`, `MemAvailable`, buffers, cached, swap, etc.

Because the contents change continuously, reading `/proc` gives a **snapshot**. A process may disappear between the time its directory is listed and the time its files are opened; the program must be prepared for this (Section 5.9).

2.5 CPU Scheduling and CPU Usage Accounting

The scheduler decides which runnable process gets a CPU core and for how long. Linux uses a

fair-share, time-sliced scheduler (the Completely Fair Scheduler, replaced by the EEVDF scheduler from Linux 6.6). As a process runs, the kernel adds the time it spends executing to two per-process counters: `utime` (time in user mode) and `stime` (time in kernel mode on behalf of the process). These counters are measured in **clock ticks**; the number of ticks per second is obtained with `sysconf(_SC_CLK_TCK)` and is usually 100.

The counters are **cumulative since the process started**, so a single reading does not tell how busy the process is *now*. CPU usage is a rate, and a rate needs two samples taken a known time apart. The method used is described in Section 6.5; in outline:

CPU % = 100 × (Δ process ticks) ÷ (Δ total system ticks)

where Δ process ticks is the increase in `utime + stime` of the process between the two samples and Δ total system ticks is the increase in the sum of all fields on the first line of `/proc/stat`. The ratio is the process's share of total machine capacity; it can be multiplied by the number of online CPUs to express usage relative to one core, which is the convention used by `top`.

2.6 Virtual Memory, Resident Memory, and RAM Utilization

Linux gives each process its own **virtual address space**. Only the pages that have actually been touched are placed in physical memory; others may be unallocated, shared, or swapped out. Three figures are therefore different:

- **Virtual size (`VmSize`):** the total address space mapped by the process. It is usually much larger than real memory use and is not a good measure of RAM consumption.
- **Resident set size (`VmRSS`):** the part of the process currently held in physical RAM. This is the figure the task manager reports because it reflects actual RAM usage.
- **System RAM (`MemTotal`, `MemAvailable`):** read from `/proc/meminfo`; used as the denominator for the memory percentage and to describe overall utilisation.

`VmRSS` includes pages shared with other processes (for example, shared libraries), so the sum of the `VmRSS` values of all processes can exceed physical memory. This is a known property of the metric and is stated as a limitation in Section 8.11.

2.7 Linux Signals and Process Termination

A **signal** is a short asynchronous notification sent to a process by the kernel or by another process. A process may install a handler for most signals, ignore them, or accept the default action. Two signals are used for termination in this project:

- **`SIGTERM` (15):** a polite request to terminate. The process can catch it, close files, save state and exit cleanly, or it may ignore it.

- **`SIGKILL` (9):** a forced termination handled by the kernel. It cannot be caught, blocked or ignored, so the process gets no chance to clean up. It is therefore used only when `SIGTERM` has failed.

Signals are sent with the `kill()` system call, whose name is misleading: it can send any signal. The call succeeds only if the sender has permission (same user ID, or root). If the target does not exist it fails with `ESRCH`; if permission is missing it fails with `EPERM`. Two special cases must be avoided: a PID of `0` or a negative PID sends the signal to a **group** of processes, which could terminate far more than the user intended. A process in state `D` may not react until its I/O completes, and a zombie (`Z`) is already dead and can be removed only by its parent.

2.8 Existing Methods and Tools, such as ps, top, and htop

Table 2.4: Comparison with existing tools

Tool	What it does	Limitation relevant to this project
`ps`	Prints a one-time snapshot of processes with selectable columns.	Not live; `%CPU` is the average since process start; no built-in control.
`top`	Interactive, continuously refreshed display of usage; can send signals.	Many keyboard commands to learn; large feature set.
`htop`	Enhanced `top` with colours, scrolling, search, tree view and kill menu.	Usually an extra package; source is larger and uses the ncurses library.
`kill` / `pkill`	Sends a signal to a PID or to processes matching a name.	Does not show usage; `pkill` patterns can match more processes than intended.

All of these tools obtain their data from the same `/proc` files used in this project. The project does not claim to be better than them; it re-implements their core idea in a compact form so that the underlying mechanism is visible and explainable.

2.9 Proposed Approach

The proposed approach is to build a single C program that (1) scans `/proc` for numeric directories, (2) parses `/proc/[PID]/stat` and `/proc/[PID]/status` for each process, (3) takes two samples of CPU counters separated by a short interval to compute CPU percentage, (4) reads `VmRSS` and `MemTotal` to compute memory percentage, (5) lets the user search for and select a PID from a menu, and (6) sends `SIGTERM` or `SIGKILL` to the selected process after validating the input and confirming the action. Every step checks return values so that permission errors and vanished processes are reported instead of crashing the program.

2.10 Advantages of the Proposed Approach

- **Unified interface:** list, search, measure and control in one program.
- **Real data:** all values come from the kernel through `/proc`; none are simulated.
- **Lightweight:** written in C with only standard library and POSIX calls; no external

libraries are required.

- **Educational transparency:** each displayed value can be traced to a specific file and field.
- **Safer control:** input is validated, dangerous PID values are rejected and errors are explained.
- **Portable across Linux systems:** relies on interfaces present in all mainstream distributions.

CHAPTER 3: SYSTEM REQUIREMENTS

Note to authors (delete before submission): *Only the hardware and software that the project actually needs are listed. Version numbers below are placeholders until the team records the versions used during development and testing.*

3.1 Hardware Requirements

The application reads small text files and prints a table, so its processing needs are modest. The hardware needed is simply a computer able to run Ubuntu Linux (a physical machine, a virtual machine, or Windows Subsystem for Linux with a real Linux kernel):

Table 3.1: Hardware used

Component	Requirement
Processor	Any CPU supported by the chosen Ubuntu release; multi-core is useful for observing CPU usage on more than one core.
Memory (RAM)	Enough for the Ubuntu environment itself; the task manager's own memory use has not been measured. [Record RAM of test machine]
Storage	Space for the Ubuntu installation, GCC toolchain and project source files.
Display / input	Keyboard and a terminal window; no graphics hardware is required.

3.2 Software Requirements

Table 3.2: Software used

Software	Purpose	Version used
Ubuntu Linux	Operating system providing `/proc` and POSIX interfaces	[e.g., Ubuntu xx.04 LTS]
GCC (GNU Compiler Collection)	Compiles C source code to an executable	[output of gcc --version]
GNU Make	Automates compilation (if a Makefile is used)	[output of make --version]
Git	Version control of source and documentation	[output of git --version]
Text editor / IDE	Writing source code	[e.g., VS Code / vim / nano]
Terminal emulator	Running the program and test commands	Default Ubuntu terminal

3.3 Operating System Environment — Ubuntu Linux

Ubuntu is a widely used Debian-based Linux distribution that provides a recent kernel with the full `/proc` interface and installs the GCC toolchain through the `build-essential` package. The program is developed and demonstrated in an Ubuntu terminal. Since `/proc` is a Linux-specific feature, the application will not work on Windows or macOS directly. On a Windows machine, Windows Subsystem for Linux 2 (WSL 2) runs a real Linux kernel and can be used, but processes visible there are those inside the Linux environment only. [State the environment actually used: native Ubuntu / virtual machine / WSL 2].

3.4 C Compiler and GCC

GCC is the standard compiler on Ubuntu. It translates the C source files into object code and links them with the C standard library to produce an executable. Warning options such as `-Wall` and `-Wextra` are used to catch common mistakes. GCC is installed with `sudo apt install build-essential`, which also installs `make` and the standard C library headers.

3.5 Development Tools

- **GCC and Make** for building.
- **Git** for tracking changes and **GitHub** for hosting the repository.
- **Standard command-line utilities** (`ps`, `top`, `grep`, `cat`, `sleep`, `kill`) for cross-checking the program's output during testing.
- Optional: **GDB** or **Valgrind** for debugging and memory-error checking, if used by the team. [Confirm whether used]

3.6 Minimum System Requirements

The project has not been benchmarked on low-end hardware, so no verified minimum figure is claimed. In practice, any machine that can run a supported Ubuntu release together with the GCC toolchain and a terminal satisfies the needs of the program. The minimum requirements published by Canonical for the chosen Ubuntu release should be used as the reference. [Optionally record the specification of the machine on which the project was actually tested].

3.7 Functional Requirements

Table 3.3: Functional requirements

ID	Requirement
FR-1	The system shall display a menu and accept the user's choice from the keyboard.
FR-2	The system shall list running processes with PID, PPID, name and state read from `/proc`.
FR-3	The system shall calculate and display per-process CPU usage using two samples of CPU counters.
FR-4	The system shall display per-process resident memory (`VmRSS`) and memory percentage of total RAM.
FR-5	The system shall allow the user to search for a process by PID or by name.
FR-6	The system shall allow the user to select a process and send `SIGTERM` or `SIGKILL` to it.
FR-7	The system shall validate user input and reject unsafe PIDs (non-numeric, zero, negative).
FR-8	The system shall report errors (permission denied, process not found) with clear messages.

3.8 Non-Functional Requirements

Table 3.4: Non-functional requirements

Attribute	Requirement
Performance	Should respond quickly enough for interactive use; scanning `/proc` involves reading a few small files per process.
Reliability	Must not crash when a process exits while being read or when a file cannot be opened.
Safety	Must not signal a process without a validated PID and, where implemented, user confirmation.
Usability	Menu-driven, plain-language prompts and messages suitable for beginners.
Portability	Runs on Linux systems providing `/proc` and POSIX interfaces.
Maintainability	Code divided into modules with a documented build process.
Low footprint	No third-party libraries; uses only the C standard library and POSIX/Linux calls.

CHAPTER 4: TECHNOLOGY AND TOOLS USED

4.1 C Programming Language

C is the language in which Linux itself is written and in which the Linux system-call interface is documented. It gives direct access to system calls, file functions and fixed-size buffers with no runtime or interpreter, which keeps the task manager small and fast. It also exposes the programmer to manual memory handling and string handling, so the program must check buffer sizes and return values carefully.

4.2 GCC Compiler

GCC compiles the C source into an executable. Typical options are `-Wall -Wextra` (enable warnings), `-g` (debug information) and `-o name` (output file name). Compiling with warnings enabled helps detect uninitialised variables, unused values and format-string errors before the program is run.

4.3 Linux /proc Filesystem

`/proc` is the data source of the whole project (Section 2.4). The program uses it purely as a set of read-only text files. No kernel module, special privilege or extra library is required to read the files of processes owned by the same user; most per-process files are world-readable, although some (for example `/proc/[PID]/io`) are restricted.

4.4 Linux System Calls and APIs

A **system call** is a controlled entry point into the kernel. The program uses standard library functions that wrap or sit on top of system calls. Table 4.1 lists the interfaces relevant to this project and their purpose; Section 6.9 states which of them appear in the final source code.

Table 4.1: Library functions and system interfaces

Function	Header	Purpose
`opendir()`, `readdir()`, `closedir()`	`<dirent.h>`	Open `/proc` and iterate over its directory entries to discover PIDs.
`fopen()`, `fgets()`, `fclose()`	`<stdio.h>`	Open and read the text files in `/proc`.
`sscanf()`, `strtol()`, `strstr()`	`<stdio.h>`, `<stdlib.h>`, `<string.h>`	Parse numbers and fields; validate that user input is a number; search names.
`sysconf()`	`<unistd.h>`	Obtain clock ticks per second (`_SC_CLK_TCK`) and number of CPUs (`_SC_NPROCESSORS_ONLN`).
`sleep()` / `usleep()` / `nanosleep()`	`<unistd.h>`, `<time.h>`	Wait between the two CPU samples.
`kill()`	`<signal.h>`	Send `SIGTERM` or `SIGKILL` to a process.

Function	Header	Purpose
`errno`, `strerror()`	`<errno.h>`, `<string.h>`	Identify and describe the reason for a failed call.

Note to authors (delete before submission): *Table 4.1 lists the interfaces the design calls for. Remove any row for a function that is not present in the final source code.*

4.5 Signals — SIGTERM and SIGKILL

`SIGTERM` is used first because it lets the target process exit cleanly. `SIGKILL` is the forceful option and is used only when the user explicitly chooses it or when `SIGTERM` has had no effect. The numeric values are 15 and 9 on Linux, but the program uses the symbolic names from `<signal.h>` for portability and readability.

4.6 Process Information Files

Table 4.2: Fields used from process files

Source	Field	Meaning
`/proc/[PID]/stat`	Field 1 (pid)	Process ID
`/proc/[PID]/stat`	Field 2 (comm)	Executable name in parentheses
`/proc/[PID]/stat`	Field 3 (state)	State character such as R, S, D, Z, T
`/proc/[PID]/stat`	Field 4 (ppid)	Parent process ID
`/proc/[PID]/stat`	Field 14 (utime)	User-mode CPU time in clock ticks
`/proc/[PID]/stat`	Field 15 (stime)	Kernel-mode CPU time in clock ticks
`/proc/[PID]/status`	`VmRSS`	Resident memory in kB
`/proc/stat`	First line `cpu ...`	System-wide cumulative CPU ticks
`/proc/meminfo`	`MemTotal`	Total usable RAM in kB

4.7 Linux Commands and Utilities

Standard utilities serve as **independent references** to check the application's output: `ps -o pid,ppid,stat,%cpu,rss,comm -p <PID>`, `top -b -n 1 -p <PID>`, `cat /proc/<PID>/status`, `free -k`, and `grep MemTotal /proc/meminfo`. Test processes are created with `sleep 600 &` or `yes > /dev/null &`. These utilities are used only for verification and are not called by the program.

4.8 Makefile and Build Process

A Makefile records how to compile and link the source files so that the whole project can be built with a single `make` command and cleaned with `make clean`. Make rebuilds only the files that changed, which saves time as the project grows. [Confirm that a Makefile exists in the project; if not, replace this section with the direct gcc command.]

4.9 Git and GitHub

Git is a distributed version control system that records every change to the source files, allowing the team to work in parallel on branches and to return to an earlier version if an error is introduced. GitHub hosts the remote repository, giving all three team members a shared copy and a history of contributions. [Add repository URL]

4.10 Other Libraries or Tools Actually Used

The design deliberately uses no third-party libraries: only the C standard library and POSIX/Linux headers listed in Table 4.1. [List any additional library, such as ncurses, only if it is really used in the source code. Otherwise keep this statement.]

CHAPTER 5: SYSTEM ARCHITECTURE AND DESIGN

> ***Note to authors (delete before submission):*** *Chapters 5 and 6 describe the design of the application as defined in the project brief. The module names and the flow shown in the diagrams must be compared with the final source code; adjust any module, box or arrow that differs.*

5.1 Overall System Architecture

The application follows a **layered, modular architecture**. At the top is the user interface, which communicates with the user through the terminal. Below it is a set of functional modules, each responsible for one task. At the bottom are the Linux interfaces: the `/proc` pseudo-filesystem for reading information and the kernel signal interface for control. A utility module for validation and error handling supports all the others. The architecture is shown in Figure 5.1.

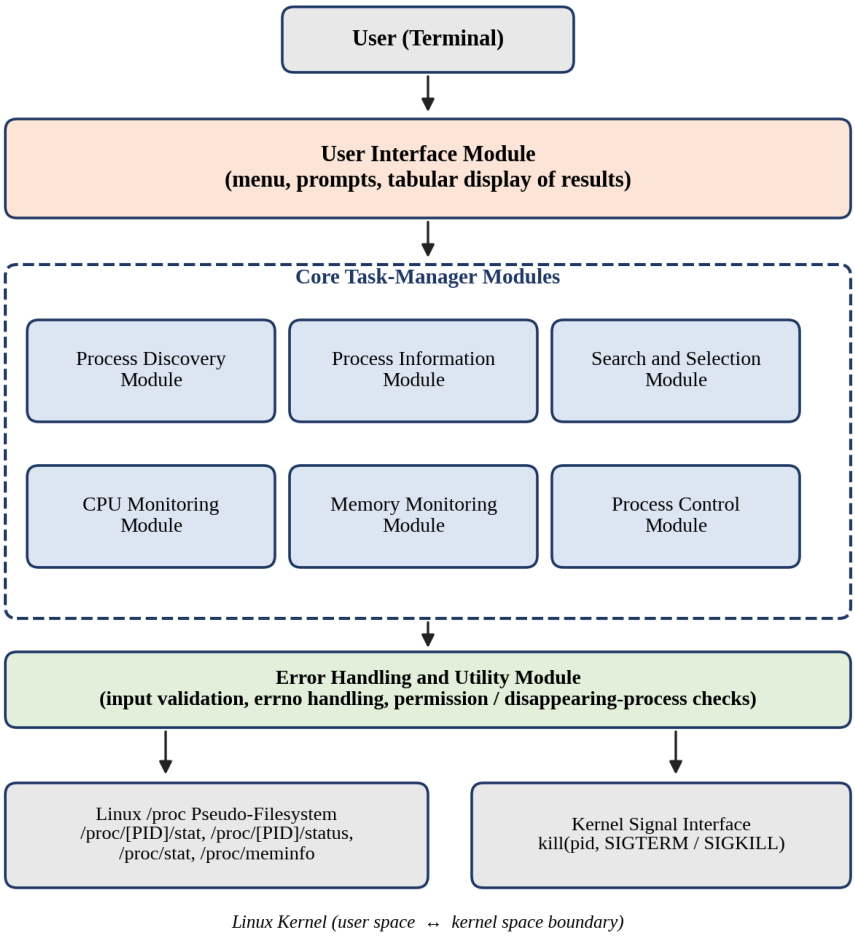


Figure 5.1: Overall system architecture of the Multi-Process Task Manager

5.2 System Workflow

The program runs as an interactive loop. At start-up it prints a menu. According to the choice made, the corresponding module reads data from `/proc`, performs any calculation, and the result is displayed. For a control action, the user supplies a PID, which is validated, and a signal is sent. After each action the program returns to the menu until the user selects Exit.

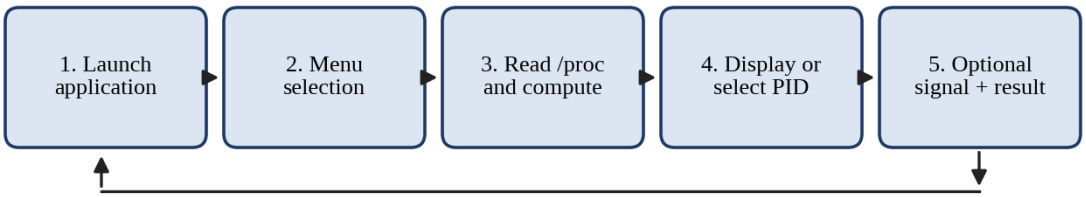


Figure 5.2: High-level workflow of the application

5.3 Architecture Diagram

Figure 5.1 is the architecture diagram. The dashed region contains the six functional modules, the green bar is the utility module used by all of them, and the two grey boxes at the bottom represent the Linux kernel interfaces with which the program communicates. Data flows downward as requests and upward as results.

5.4 Data Flow Diagram

Figure 5.3 shows the data flow. The user supplies menu choices, PIDs and search keywords to process 1.0, which validates them. Process 2.0 reads data stores D1 (`/proc/[PID]/` files) and D2 (`/proc/stat` and `/proc/meminfo`) and computes CPU and memory percentages. Process 3.0 formats the results for display. Process 4.0 sends a signal through the kernel interface (D3) and returns a success or error message to the user.

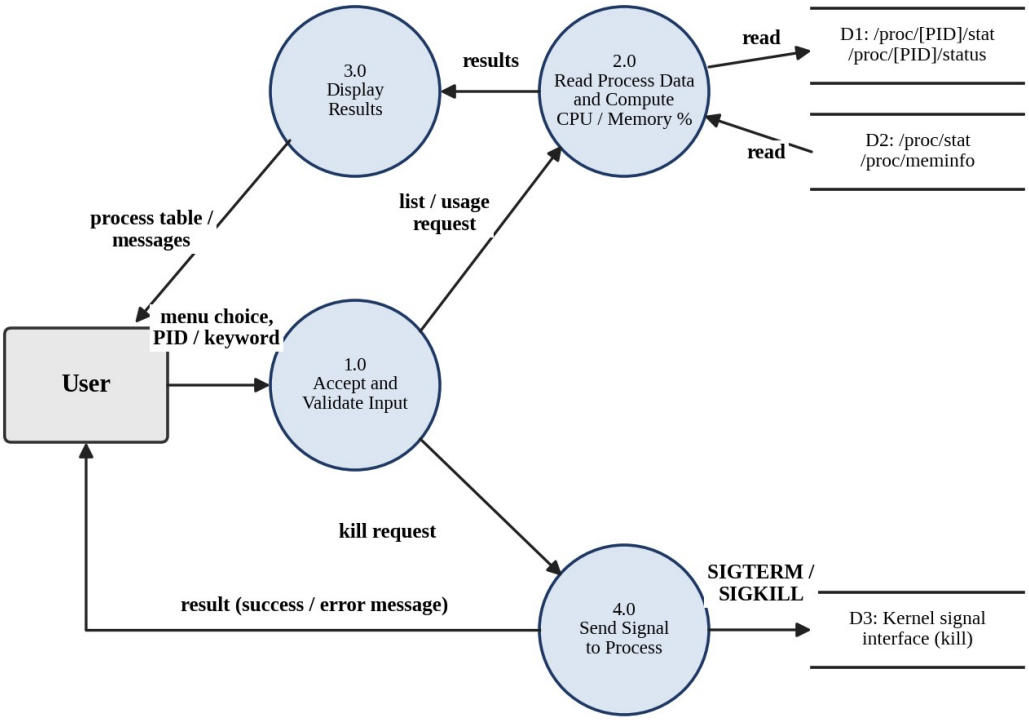


Figure 5.3: Data flow diagram of the task manager

5.5 Process Monitoring Flowchart

Figure 5.4 shows the steps performed when the user chooses to view processes and their usage. Notice that the CPU percentage requires a waiting interval between two samples, and that any process that vanishes during the scan is skipped instead of stopping the program.

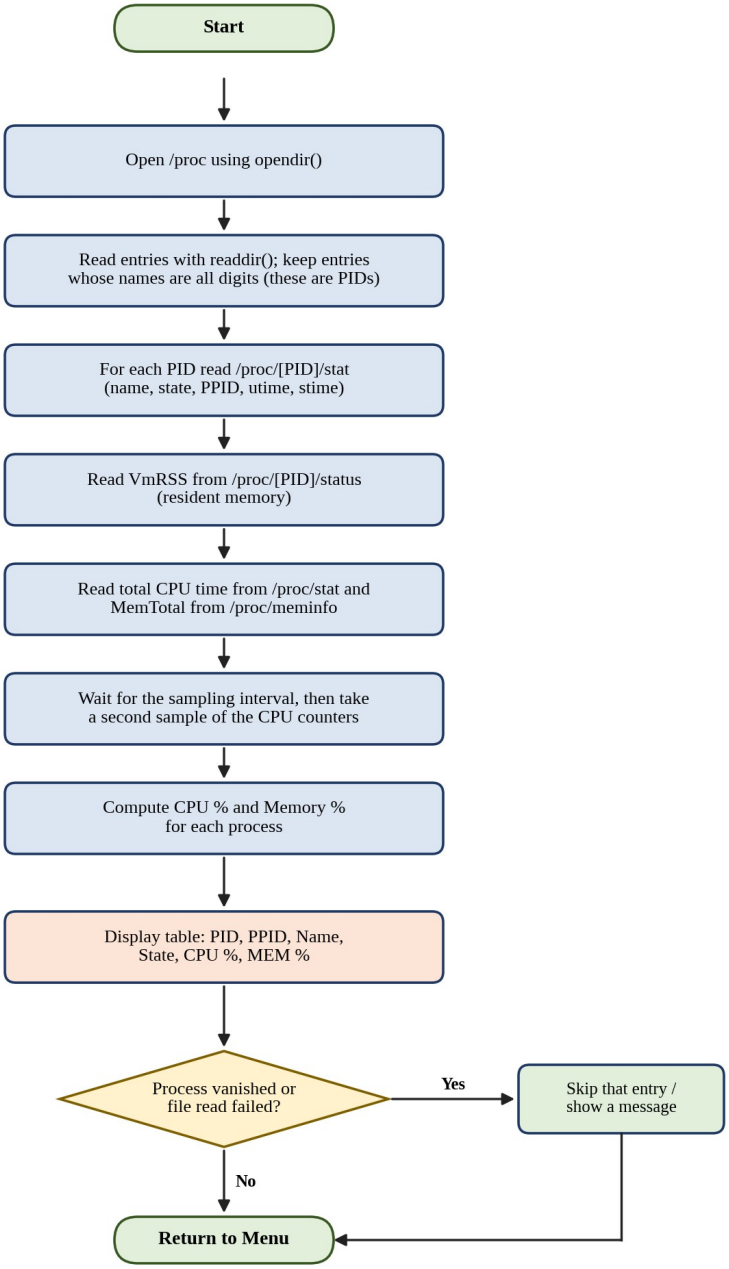


Figure 5.4: Flowchart of process monitoring

5.6 Process Termination Flowchart

Figure 5.5 shows the safety checks performed before and after a signal is sent. The input is validated as a positive integer, the existence of the process is checked, the user confirms, and only then is `SIGTERM` sent. If the process does not terminate, the user may choose the forceful `SIGKILL` signal.

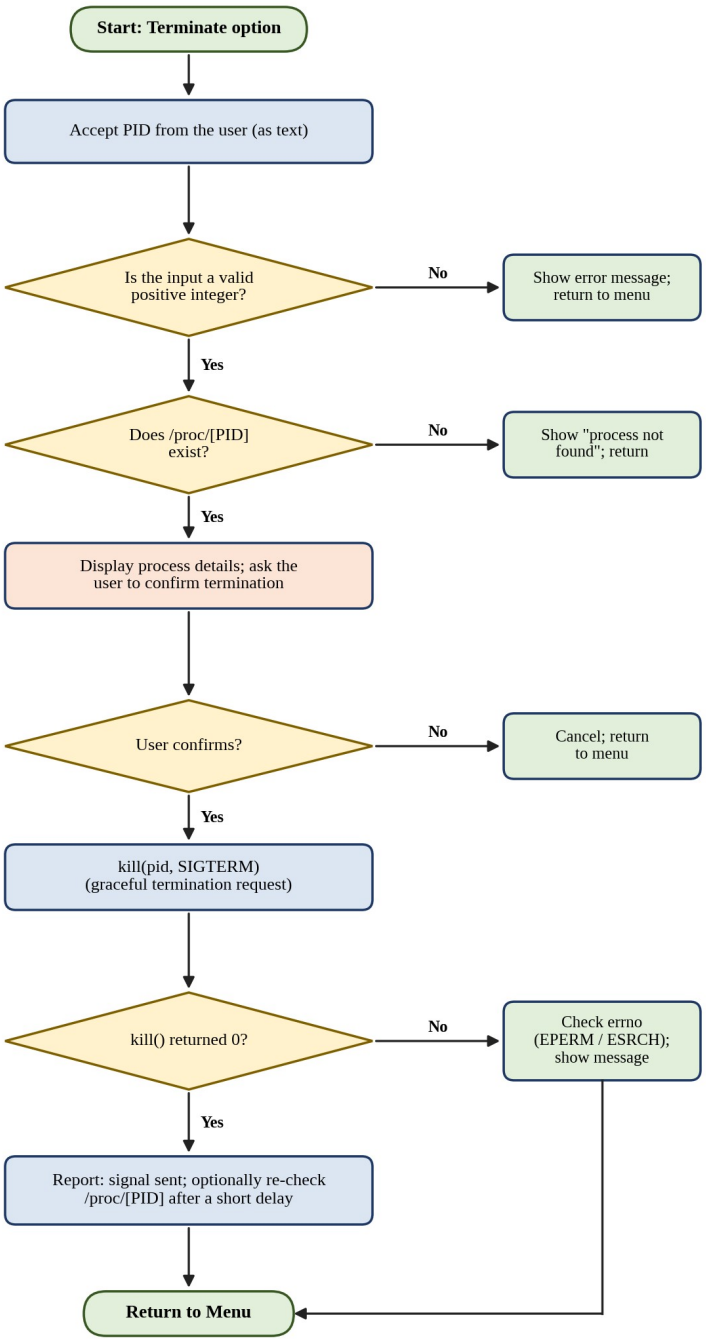


Figure 5.5: Flowchart of safe process termination

5.7 Module Description

Table 5.1: Modules and their responsibilities

Module	Responsibility	Main OS interface
User Interface Module	Displays the menu, reads the choice, prints the process table and messages.	Standard input/output
Process Discovery Module	Scans `/proc` and collects every directory name made only of digits as a PID.	`opendir()`, `readdir()`
Process Information Module	Reads name, state and PPID from `/proc/[PID]/stat` (and `status`).	`fopen()`, `fgets()`, `sscanf()`

Module	Responsibility	Main OS interface
CPU Monitoring Module	Reads `utime`, `stime` and system totals twice; computes CPU %.	`/proc/stat`, `sysconf()`
Memory Monitoring Module	Reads `VmRSS` and `MemTotal`; computes memory %.	`/proc/[PID]/status`, `/proc/meminfo`
Process Search and Selection Module	Finds processes by PID or name and lets the user choose one.	String functions
Process Control Module	Sends `SIGTERM` / `SIGKILL` to the selected PID and interprets the result.	`kill()`, `errno`
Error Handling and Utility Module	Validates input, converts `errno` to messages, provides helper functions.	`strtol()`, `strerror()`

5.8 Input and Output Design

Input is entirely from the keyboard: a menu number, a PID, or a search keyword. All input is read as text and converted with explicit checks, because reading a number directly can leave the program in an undefined state when the user types letters. **Output** is a formatted text table in the terminal with the columns PID, PPID, NAME, STATE, CPU % and MEM %, followed by status or error messages. Table 5.2 summarises the design.

Table 5.2: Input and output design

Item	Type	Validation / format
Menu choice	Integer	Must be within the range of the menu
PID	Positive integer	All digits, greater than zero, fits in the PID type
Search keyword	Text	Non-empty; length-limited; matched against process names
Confirmation	Character (y/n)	Only the expected answers are accepted
Process table	Text table	PID, PPID, NAME, STATE, CPU %, MEM %
Messages	Text	Success, “not found”, “permission denied”, “invalid input”

5.9 Error Handling and Permission Management

Because `/proc` is dynamic, errors are expected during normal operation. Table 5.3 lists the main error situations and the planned response.

Table 5.3: Error situations and responses

Situation	Detection	Response
Process exits while being read	`fopen()` returns NULL with `ENOENT`/`ESRCH`	Skip the process silently or show a short note; continue the scan
No permission to read a file	`fopen()` fails with `EACCES`	Display “N/A” for that field; continue
No permission to signal	`kill()` returns -1, `errno == EPERM`	Explain that the process belongs to another user or to root

Situation	Detection	Response
PID does not exist	`kill()` returns -1, `errno == ESRCH`	Report “process not found”
Invalid user input	Failed numeric check	Print message and ask again or return to the menu
Unexpected file format	`sscanf()` returns fewer fields than expected	Skip the process and continue

Ordinary users may signal only their own processes. The program does **not** attempt to bypass this restriction. Running it with `sudo` extends reach to all processes, which is why the safety checks of Section 5.10 matter.

5.10 Security Considerations

- **Never trust the PID:** only a positive integer is accepted. `0` and negative values are rejected because `kill()` treats them as process-group targets.
- **Confirm destructive actions:** the user sees the process name before it is signalled.
- **Prefer `SIGTERM`:** `SIGKILL` gives the target no chance to save data and is offered as a second step.
- **Protect critical processes:** extra care is advisable for PID 1 and for the task manager’s own PID. [Confirm whether the program blocks these.]
- **No privilege escalation:** the program does not set the set-user-ID bit and does not call `sudo` internally.
- **Safe string handling:** use bounded functions (`fgets`, `snprintf`) to avoid buffer overflows when reading names and file lines.

CHAPTER 6: IMPLEMENTATION

Note to authors (delete before submission): *The source code was not available when this report was prepared. Sections that depend on real file names, function names or code listings are marked with highlighted placeholders. Replace them with material from the final source and delete this note.*

6.1 Project Directory Structure

The directory structure below shows the arrangement recommended for the project. The names must be replaced by the real files of the repository.

```
Multi_Process_Task_Manager/
|-- [main.c]            (program entry point, menu loop)
|-- [process.c / .h]    (process discovery and information)
|-- [cpu.c / .h]        (CPU usage calculation)
|-- [memory.c / .h]     (memory usage calculation)
|-- [control.c / .h]    (search, selection, signal sending)
|-- [utils.c / .h]      (validation and error helpers)
|-- Makefile              ([if present])
|-- README.md
`-- docs/                 (report, screenshots)
```

6.2 Module-Wise Implementation

Each module of Table 5.1 is implemented as a group of functions with a clear input and output. The modules do not share global state except for the process table structure that holds the collected data. A typical record for a process contains the PID, PPID, name, state, the two CPU time values, the resident memory and the calculated percentages. [Insert the actual struct name and members from the source.]

6.3 Process Discovery Using /proc

Discovery works by listing the directory `/proc`. For each entry returned by `readdir()`, the program checks whether the name consists only of decimal digits. Names like `cpuinfo`, `meminfo`, `self` or `sys` are not numeric and are ignored; the numeric ones are PIDs of the processes that existed at the time of the scan. The directory stream is closed with `closedir()` after the scan to avoid leaking the file descriptor.

6.4 Reading Process Information

For each PID the program opens `/proc/[PID]/stat` and reads its single line with `fgets()`. This line has the form `pid (name) state ppid ...`. The command name is enclosed in parentheses and **may itself contain spaces or parentheses** (for example `(Web Content)`), so a naive split on spaces is unreliable. The safe method is to locate the first `(` and the **last** `)` in the line, take the text between them as the name, and parse the remaining fields after the last `)`. From these fields the program takes the state (field 3), PPID (field 4), `utime` (field 14) and `stime` (field 15).

6.5 CPU Usage Calculation

CPU usage is computed from two samples taken a fixed interval apart (for example one second):

1. **Sample 1:** for each process read `utime + stime` (call it `P1`); read the first line of `/proc/stat` and add all its fields to obtain the total system ticks `T1`.
2. **Wait** for the sampling interval.
3. **Sample 2:** repeat the readings to obtain `P2` and `T2`.
4. **Compute:** `CPU% = 100 × (P2 − P1) ÷ (T2 − T1)`. Multiplying by the number of online CPUs, obtained with `sysconf(_SC_NPROCESSORS_ONLN)`, gives the per-core convention used by `top`, in which a single-threaded busy loop shows about 100%.

The number of clock ticks per second from `sysconf(_SC_CLK_TCK)` is needed only when converting ticks to seconds; in the ratio above the unit cancels. [State which convention (share of whole machine or per-core) the program prints, and whether the sample interval is fixed.]

Limitations of CPU sampling. The result is an average over the interval, so very short bursts can be missed or smoothed. A process that starts or ends between the two samples has only one reading and must be skipped or shown as 0. The first display has no "previous" sample, so it requires an extra waiting period. Tick counters have a resolution of about 10 ms, which limits accuracy for small intervals. Finally, the values from `ps` (average since start) and from `top` (average since its last refresh) are calculated differently, so exact agreement with them should not be expected.

6.6 Memory Usage Calculation

For each process the program reads the line beginning with `VmRSS:` in `/proc/[PID]/status`; the value is in kilobytes. Kernel threads have no user address space and therefore have no `VmRSS` line; such processes are shown with 0 or "N/A". The total RAM is read once from the `MemTotal:` line of `/proc/meminfo`. The memory percentage is:

MEM% = 100 × VmRSS ÷ MemTotal

`MemAvailable` from the same file can be used to report overall system utilisation as `(MemTotal − MemAvailable) ÷ MemTotal`. As explained in Section 2.6, `VmRSS` counts shared pages in every process that maps them, so per-process percentages may add up to more than 100%.

6.7 Process Search and Selection

Searching is done on the list that has already been collected in memory, so no new reads of `/proc` are needed. A numeric keyword is compared with PIDs; a text keyword is matched as a substring of the process name. All matching processes are displayed, and the user then enters the

PID of the one to be acted on. The selected PID is re-validated at the time of use because the process list may be a few seconds old.

6.8 Safe Process Termination

Termination is carried out by the Process Control Module in the following order: (1) validate that the PID is a positive integer; (2) check that `/proc/[PID]` still exists; (3) display the process name and ask the user for confirmation; (4) call `kill(pid, SIGTERM)`; (5) inspect the return value and `errno`; (6) optionally allow the user to escalate to `SIGKILL`.

Process permissions. If the target belongs to another user, `kill()` fails with `EPERM`. The program reports this and does not retry.

PID reuse. Between the moment the user views the table and the moment the signal is sent, the original process may have exited and the kernel may have assigned the same PID to a different new process. The risk is small because PIDs are allocated cyclically up to `pid_max`, but it is real. It is reduced by keeping the interval between selection and signalling short, by re-checking the process name immediately before sending the signal, and by asking the user to confirm. It cannot be removed completely with a plain PID; Linux offers `pidfd_open()` for race-free handling, which is noted as future work.

Zombies and uninterruptible processes. A zombie cannot be killed, because it is already dead; its parent must collect it. A process in state `D` may not respond until its I/O completes. The program reports that the signal was *sent*, which is not the same as the process having *terminated*; for that reason the termination flowchart includes an optional re-check.

6.9 Linux System Calls and APIs Used

Table 6.1 relates each interface to the module that uses it. [Delete rows for functions that are not in the final source code.]

Table 6.1: Interfaces used by each module

Module	Interfaces
Process Discovery	`opendir()`, `readdir()`, `closedir()`
Process Information	`fopen()`, `fgets()`, `fclose()`, `sscanf()`
CPU Monitoring	`fopen()`, `fgets()`, `sscanf()`, `sysconf()`, `sleep()`/`usleep()`
Memory Monitoring	`fopen()`, `fgets()`, `sscanf()`/`strncmp()`
Search and Selection	`strtol()`, `strstr()`
Process Control	`kill()`, `errno`, `strerror()`
User Interface / Utility	`printf()`, `fgets()`/`scanf()`, `snprintf()`

6.10 Input Validation and Error Handling

All numeric input is converted with `strtol()`, which reports where parsing stopped and whether the value overflowed. The program accepts the input only if the whole string was consumed, no error occurred and the value is within the valid range. Every call that can fail (`fopen`, `opendir`, `kill`) is followed by a test of its return value, and `errno` is read immediately, before any other call can overwrite it. Error messages are produced in one place (the utility module) so that the wording is consistent.

6.11 Algorithms and Pseudocode

Algorithm 1: Process discovery and information

```
function collect_processes():
    dir <- opendir("/proc")
    for each entry in dir:
        if entry.name is all digits:
            pid <- to_integer(entry.name)
            if read_stat(pid, name, state, ppid, utime, stime) fails:
                continue                    # process vanished
            rss_kb <- read_vmrss(pid)       # 0 if absent
            add (pid, ppid, name, state, utime+stime, rss_kb) to table
    closedir(dir)
    return table
```

Algorithm 2: CPU and memory percentage

```
function compute_usage(table):
    T1 <- total_cpu_ticks("/proc/stat")
    snapshot1 <- table.cpu_ticks           # utime + stime per PID
    wait(interval)
    refresh table                           # second sample
    T2 <- total_cpu_ticks("/proc/stat")
    mem_total <- read_memtotal("/proc/meminfo")
    for each process p present in both samples:
        p.cpu_pct <- 100 * (p.ticks2 - snapshot1[p]) / (T2 - T1)
        p.mem_pct <- 100 * p.rss_kb / mem_total
```

Algorithm 3: Safe termination

```
function terminate(pid_text):
    if not is_positive_integer(pid_text): error("invalid PID"); return
    pid <- to_integer(pid_text)
    if not exists("/proc/" + pid):        error("process not found"); return
    show name of pid; if not confirm():   return
    if kill(pid, SIGTERM) == 0:           report("SIGTERM sent")
    else if errno == EPERM:               error("permission denied")
    else if errno == ESRCH:               error("process not found")
    optionally: if user requests, kill(pid, SIGKILL)
```

6.12 Core C Code Explanation

This section explains the actual functions of the program. Because the source files were not supplied with the template, the code listings must be added by the team. For each of the functions in Table 6.2, paste the real code (monospaced) and explain it in two or three sentences:

what it reads, what it returns, and how it handles failure.

Table 6.2: Functions to be documented from the source code

Task	Function name in source	Source file	Listing
Scan `/proc` for PIDs	[function]	[file]	Listing 6.1
Parse `/proc/[PID]/stat`	[function]	[file]	Listing 6.2
Read `VmRSS`	[function]	[file]	Listing 6.3
Read total CPU ticks	[function]	[file]	Listing 6.4
Compute CPU % and MEM %	[function]	[file]	Listing 6.5
Search / select process	[function]	[file]	Listing 6.6
Send signal with `kill()`	[function]	[file]	Listing 6.7

Listing 6.1 onwards: [insert real C code excerpts here]

6.13 Compilation and Build Process

The project is built on Ubuntu with GCC. Either of the following commands may be used; keep the one that matches the repository.

```
# Option 1: using the Makefile (if present)
make

# Option 2: direct compilation
gcc -Wall -Wextra -g -o [task_manager] [*.c]

# Remove build output
make clean
```

During compilation, GCC preprocesses the source (expanding `#include` and macros), compiles each `.c` file to an object file, and the linker combines the object files with the C library into the executable. Warnings must be fixed rather than ignored.

CHAPTER 7: EXECUTION AND DEMONSTRATION

> ***Note to authors (delete before submission):*** *This chapter gives the commands and procedure for running the project. Screenshots must be captured by the team from the real program and inserted into the marked boxes. No output is shown here that has not been produced by the program.*

7.1 Ubuntu Environment Setup

Open a terminal and install the build tools and Git, then check their versions:

```
sudo apt update
sudo apt install build-essential git

gcc --version
make --version
git --version
```

Obtain the source code and move into the project directory:

```
git clone [repository URL]
cd [project folder]
```

7.2 Compilation Using GCC or Makefile

```
make
# or
gcc -Wall -Wextra -g -o [task_manager] [*.c]
```

A successful build produces the executable in the project directory without errors. [Mention any warnings remaining.]

7.3 Execution Commands

```
./[task_manager]          # run as a normal user (own processes can be controlled)
sudo ./[task_manager]     # optional: run with elevated rights to reach all
processes
```

For testing, create harmless processes in a second terminal window:

```
sleep 600 &                  # an idle test process
yes > /dev/null &            # a CPU-intensive test process (stop it afterwards)
python3 -c "b=bytearray(100*1024*1024); import time; time.sleep(300)" &   # about
100 MB of memory
```

7.4 Application Menu

On start-up the program shows a menu. The options expected from the design are listed in Table 7.1. [Replace with the exact menu text and numbering of the program.]

Table 7.1: Application menu options

Option	Function
1	Display running processes
2	Search for a process (by PID or name)

Option	Function
3	View CPU and memory usage
4	Terminate a selected process
5	Exit



Figure 7.1: Application main menu

7.5 Displaying Running Processes

Choosing the display option lists the processes with PID, PPID, name and state. The values can be compared with `ps -eo pid,ppid,stat,comm`.



Figure 7.2: List of running processes

7.6 Searching for a Process

Start `sleep 600 &`, then search for the keyword `sleep` or for the PID printed by the shell. The matching entries should be displayed.



Figure 7.3: Searching for a process by name or PID

7.7 Viewing CPU and Memory Usage

With the `yes` and memory test processes running, view the usage option. The CPU figure of `yes` is expected to be high, and the memory figure of the Python process should correspond to roughly 100 MB. Compare them with `top -b -n 1 -p <PID>` and `grep VmRSS /proc/<PID>/status`.

[SCREENSHOT PLACEHOLDER]

Insert screenshot of CPU and memory usage (replace this box with the real screenshot)

Figure 7.4: CPU and memory usage display

[SCREENSHOT PLACEHOLDER]

Insert screenshot of the reference command (replace this box with the real screenshot)

Figure 7.5: Verification of the same process using top / ps

7.8 Terminating a Selected Test Process

Select the test PID in the terminate option and confirm. Then verify with `ps -p <PID>` or by listing the processes again; the entry should be gone. Only test processes created for this purpose should be terminated.

[SCREENSHOT PLACEHOLDER]

Insert screenshot of termination (replace this box with the real screenshot)

Figure 7.6: Terminating a test process (before and after)

7.9 Error Handling Demonstration

Demonstrate (a) non-numeric input such as `abc`, (b) PID `0` or a negative PID, (c) a PID that does not exist, and (d) a PID owned by root (for example `1`) when running as a normal user. Each case should produce a clear message and return to the menu.

[SCREENSHOT PLACEHOLDER]

Insert screenshot of error handling (replace this box with the real screenshot)

Figure 7.7: Invalid input and permission error messages

7.10 Screenshots of Execution

All screenshots required by the previous sections are inserted at the marked locations (Figures 7.1 to 7.7). Each screenshot should show the terminal prompt and command so that the evaluator can see the real execution. [Add further screenshots if needed.]

7.11 Demonstration of Operating Systems Concepts

Table 7.2: OS concepts and where they are demonstrated

OS concept	Demonstration in the application
Process and PID	Process list shows many concurrently existing processes, each with a unique PID.
Parent-child relationship	The PPID column shows the parent; the test processes started from the shell show the shell as their parent.
Process states	Busy test process appears as `R`, the `sleep` process as `S`.
CPU accounting	The `yes` process shows high CPU %, idle processes show near 0%.
Virtual vs. resident memory	Memory column reports `VmRSS`, the physical memory actually used.
Signals	`SIGTERM` terminates `sleep`; permission errors demonstrate kernel protection.
/proc as kernel interface	Every figure can be traced to a file in `/proc`.

CHAPTER 8: TESTING AND RESULTS

Note to authors (delete before submission): *No test has been recorded as executed in this report. The "Actual Output" and "Status" columns are intentionally marked pending verification. After running each test, replace them with the observed result and Pass / Fail.*

8.1 Testing Methodology

Testing is **black-box and comparative**. For each feature, a controlled test process is created, the application is used, and its output is compared with an independent reference (`ps`, `top`, `/proc` files read manually, `free`). Because process data changes continuously, small differences are expected and a tolerance is used (for example ±5 percentage points for CPU and ±5% for memory). Each test is recorded with a unique ID, the procedure, the expected result, the actual result and a status.

8.2 Functional Testing

Functional tests check that each menu option works and returns to the menu (TC-01 to TC-04 in Table 8.1): start-up, process listing, search and exit.

8.3 CPU Usage Calculation Testing

A process running `yes > /dev/null` keeps one core busy and is compared with `top`. An idle `sleep` process should show about 0%. Tests TC-05 and TC-06 cover this.

8.4 Memory Usage Testing

A process that allocates a known amount of memory is compared with the `VmRSS` line in `/proc/[PID]/status`. Tests TC-07 and TC-08 cover this.

8.5 Process Control Testing

Tests TC-09 to TC-11 check termination with `SIGTERM`, cancellation, and the forced `SIGKILL` option.

8.6 Invalid Input Testing

Tests TC-12 to TC-15 check letters, empty input, zero or negative PIDs and very large numbers.

8.7 Permission Error Testing

Tests TC-16 and TC-17 check that signalling a process owned by root as a normal user fails gracefully with a permission message.

8.8 Process Disappearance and Race Condition Testing

Tests TC-18 to TC-20 check processes that exit during a scan or between selection and signalling, and a zombie process.

8.9 Test Case Table

Table 8.1: Test cases

ID	Description	Procedure / Input	Expected Output	Actual Output	Status
TC-01	Start application	Run the executable	Menu is displayed	Pending verification	Pending
TC-02	List processes	Choose list option	Table with PID, PPID, name, state; matches `ps`	Pending verification	Pending
TC-03	Search by name	`sleep 600 &`; search “sleep”	Entry for the test process is shown	Pending verification	Pending
TC-04	Exit	Choose Exit	Program ends without errors	Pending verification	Pending
TC-05	CPU of busy process	`yes > /dev/null &`; view usage	High CPU, close to `top` value	Pending verification	Pending
TC-06	CPU of idle process	View usage of `sleep` process	Approximately 0%	Pending verification	Pending
TC-07	Memory of known process	Python process holding ~100 MB; view usage	Resident memory close to `VmRSS`	Pending verification	Pending
TC-08	Memory percentage	Compare with `VmRSS / MemTotal`	Matches within tolerance	Pending verification	Pending
TC-09	Terminate with SIGTERM	Select `sleep` PID; confirm	Process ends; success message	Pending verification	Pending
TC-10	Cancel termination	Select PID; answer “no”	No signal sent; process still alive	Pending verification	Pending
TC-11	Force with SIGKILL	Start `sleep`; use forced option	Process ends	Pending verification	Pending
TC-12	Letters as PID	Enter `abc`	“Invalid input” message	Pending verification	Pending
TC-13	Empty input	Press Enter only	Message; no crash	Pending verification	Pending
TC-14	Zero / negative PID	Enter `0` and `-1`	Rejected; no signal sent	Pending verification	Pending
TC-15	Very large number	Enter `99999999999999`	Rejected as invalid	Pending verification	Pending
TC-16	Signal root process as user	Enter PID of a root process	Permission-denied message	Pending verification	Pending
TC-17	Read restricted file	View info for a root process	Fields shown as N/A; no crash	Pending verification	Pending
TC-18	Process exits during scan	Run many short `sleep 1` processes while listing	Scan completes without crash	Pending verification	Pending
TC-19	Process exits before signal	Select PID of `sleep 2`; wait; confirm	“Process not found”	Pending verification	Pending
TC-20	Zombie process	Create zombie; try to terminate	Signal accepted but zombie remains; reported correctly	Pending verification	Pending

8.10 Results and Discussion

[Summarise the results after the tests have been run: number of tests passed, any failures, and how close the CPU and memory values were to the reference tools. Do not state results here until they have been observed.]

The expected behaviour is that the CPU figures follow the reference tools within sampling tolerance, the memory figures agree with `VmRSS`, and every error case is handled with a message instead of a crash.

8.11 Limitations

- **Sampling accuracy:** CPU usage is an average over one interval and depends on the interval length; very short bursts may be missed.
- **Snapshot nature of `/proc`:** data are not read atomically; processes can start or end during a scan.
- **PID reuse:** a PID may refer to a different process by the time a signal is sent (Section 6.8).
- **Shared memory:** `VmRSS` includes shared pages, so per-process memory may overlap.
- **Permissions:** only the user's own processes can be controlled unless run with elevated rights; some `/proc` files are unreadable for other users.
- **Zombie and `D`-state processes:** signals may have no visible effect.
- **Text interface only:** no graphical display, history, sorting or refresh control in the current version.
- **Environment:** tested only on the systems listed in Chapter 3. [Confirm]

CHAPTER 9: CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Project Summary

The Multi-Process Task Manager for Linux is a lightweight C application that reads real process information from the `/proc` filesystem, computes CPU usage from successive samples and memory usage from `VmRSS` and `MemTotal`, supports searching and selecting processes, and controls them with `SIGTERM` and `SIGKILL` under input validation and error handling. The report documents the design, the algorithms, the build process and the test plan for the project.

9.2 Objectives Achieved

Table 9.1: Objectives and status

Objective	Status
Study of Linux process representation and `/proc`	Covered in Chapter 2
Read real process data in C	Designed in Chapters 5-6; [confirm against source]
CPU and memory calculation	Method documented in Sections 6.5 and 6.6; [confirm results after testing]
Search, select and terminate processes	Designed in Sections 6.7-6.8; [confirm]
Error and permission handling	Designed in Sections 5.9 and 6.10; [confirm]
Build and documentation using GCC, Make and Git	Chapters 4, 6 and 7

[After testing, update the status column with the verified outcome of each objective.]

9.3 Operating Systems Learning Outcomes

- Understanding of the process abstraction, PID/PPID relationships and process states.
- Understanding of how the kernel accounts CPU time and how usage is derived from counters.
- Understanding of virtual memory and the meaning of resident memory.
- Practical use of system interfaces: directory and file access, `sysconf()`, `kill()` and `errno`.
- Understanding of signals, permissions and the user/kernel boundary.
- Experience with GCC, Make, Git and technical documentation.

9.4 Benefits of the Application

The application offers a single, simple interface for tasks that otherwise need several commands, helps users avoid mistyped PIDs, and serves as a teaching aid that exposes the working of tools like `ps` and `top`. It has no dependencies, so it can be built quickly on any Ubuntu system.

9.5 Current Limitations

The current limitations are those given in Section 8.11: sampling-based CPU values, snapshot

reads of `/proc`, PID reuse risk, shared-memory overlap, permission limits and a text-only interface.

9.6 Future Enhancements

The following are suggestions for future work and are **not** part of the current implementation unless the source code shows otherwise:

- A graphical interface (for example with GTK) or an ncurses-based live display.
- History of CPU and memory usage with graphs.
- Sorting and filtering of the process list by CPU, memory, PID or name.
- Configurable refresh interval.
- Resource alerts when a process exceeds a threshold.
- Process-tree visualisation using PPID relationships.
- Race-free signalling with `pidfd_open()` and `pidfd_send_signal()`.
- Per-process disk and network statistics.

CHAPTER 10: REFERENCES

10.1 Books

[1] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. San Francisco, CA, USA: No Starch Press, 2010.

[2] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

[4] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.

[5] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

10.2 Linux Manual Pages

[6] *proc(5)* - process information pseudo-filesystem, Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man5/proc.5.html

[7] *proc_pid_stat(5)* - /proc/[pid]/stat, Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/

[8] *kill(2)* - send signal to a process, Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man2/kill.2.html

[9] *signal(7)* - overview of signals, Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/man7/signal.7.html

[10] *ps(1)* and *top(1)* - procps-ng utilities, Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/

[11] *sysconf(3)*, *opendir(3)*, *readdir(3)* and *errno(3)*, Linux man-pages project. [Online]. Available: https://man7.org/linux/man-pages/

10.3 GNU and Kernel Documentation

[12] GNU Project, *GCC, the GNU Compiler Collection - Online Documentation*. [Online]. Available: https://gcc.gnu.org/onlinedocs/

[13] GNU Project, *GNU Make Manual*. [Online]. Available: https://www.gnu.org/software/make/manual/

[14] The Linux Kernel documentation, *The /proc Filesystem*. [Online]. Available: https://www.kernel.org/doc/html/latest/filesystems/proc.html

10.4 Online Resources

[15] S. Chacon and B. Straub, *Pro Git*, 2nd ed. [Online]. Available: https://git-scm.com/book

[16] Ubuntu Documentation, *Ubuntu*. [Online]. Available: https://ubuntu.com/